



RÉPUBLIQUE
FRANÇAISE

*Liberté
Égalité
Fraternité*



UNIVERSITÉ
TOULOUSE III
PAUL SABATIER

FACULTÉ SCIENCES
ET INGÉNIERIE

Base de la POO en Python

Mathieu RAYNAL

mathieu.raynal@irit.fr

<http://www.irit.fr/~Mathieu.Raynal>



Déclarer une classe

- Déclarer une classe

```
class MaClasse:  
    "Description de ma classe ..."  
  
    ...
```

Constructeur

- Constructeur

```
def __init__(self [,params]):  
    ...
```

- Il ne peut y avoir qu'un seul constructeur dans la classe
 - Possibilité de mettre des valeurs par défaut aux paramètres

```
def __init__(self, x=0, y=0):  
    ...
```

Méthode de classe

- Pour « simuler » d'autres constructeurs

```
@classmethod  
def nom_de_la_methode(cls[, params]):  
    return cls(...)
```

Attributs et méthodes d'une classe

- Attribut (variable d'instance)
 - Généralement déclaré et initialisé dans le constructeur

```
self.monAttribut = saValeur
```

- Les méthodes se déclarent comme les fonctions
 - Obligatoirement un premier paramètre nommé ***self***

```
def maMethode(self [,params]):
```

Objet : Création d'une instance d'une classe

```
class Point:  
    "Définition d'un point en 2D"
```

```
    def __init__(self, x=0, y=0):  
        self.x = x  
        self.y = y
```

```
    @classmethod  
    def creer_point(cls, y):  
        return cls(0, y)
```

```
    def deplacer(self, dx, dy):  
        self.x += dx  
        self.y += dy
```

```
p = Point(1, 2)  
p2 = Point()  
p3 = Point.creer_point(4)
```

Utilisation d'un membre depuis un objet

- Membre = attribut ou méthode

```
p.x = 5  
p2.déplacer(3, 4)
```

- Afficher la description d'un objet

```
print(p. __doc__)
```

Les méthodes `__str__` et `__repr__`

- Pour afficher la représentation d'un objet, il faut définir une des méthodes suivantes

```
def __str__(self):
```

- Ou

```
def __repr__(self):
```

- Ces méthodes doivent retourner la chaîne de caractères que l'on souhaite afficher

Les énumérations

- Déclaration d'une énumération

```
from enum import Enum, auto

class Color(Enum):
    RED = auto()
    GREEN = auto()
    BLUE = auto()
```

- Utilisation d'une énumération

```
c = Color.BLUE
if c == Color.BLUE:
    print('OK')
```

- Parcours des valeurs d'une énumération

```
for c in Color:
    print(f"{c.name} a pour valeur {c.value}")
```

Match / case

```
day = "Monday"

# Match the day to predefined patterns
match day:
    case "Saturday" | "Sunday":
        print(f"{day} is a weekend.")
    case "Monday" | "Tuesday" | "Wednesday" | "Thursday" | "Friday":
        print(f"{day} is a weekday.")
    case _:
        print("That's not a valid day of the week.")
```